

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САПР



Практична робота №2

з дисципліни: “Застосування систем штучного інтелекту у технологічних рішеннях”

на тему:

“Вибір архітектури нейромережі для прогнозу біржових котирувань.”

Виконав:

Ст. групи ПП-44

Верещак Б. О.

Прийняв:

доц. Мокрицька О. В.

Мета роботи

Ознайомити студентів з принципами вибору архітектури нейронних мереж для прогнозування часових рядів, розвинути навички підготовки даних, налаштування моделей та оцінки їх якості.

Теоретичні відомості

Прогнозування біржових котирувань є складним завданням через високу волатильність фінансових ринків і вплив численних економічних, політичних та психологічних факторів. Традиційні методи аналізу (статистичні моделі, лінійна регресія, ARIMA) мають обмежену здатність враховувати нелінійні залежності та складні взаємозв'язки у фінансових даних. Застосування нейронних мереж дозволяє автоматично виявляти складні закономірності та покращує точність прогнозів.

Типи нейронних мереж для прогнозування біржових котирувань

1. Багат шарові перцептрони (MLP)

- Використовуються для прогнозування часових рядів із фіксованою кількістю вхідних змінних.
- Мережа складається з вхідного шару, одного або декількох прихованих шарів і вихідного шару.
- Перевага: простота архітектури та ефективність для невеликих наборів даних.
- Недолік: обмежена здатність обробляти послідовності даних із довгостроковими залежностями.

2. Рекурентні нейронні мережі (RNN)

- Відрізняються наявністю циклічних зв'язків, що дозволяє враховувати попередній стан при прогнозуванні.
- Підходять для часових рядів та послідовностей котирувань.
- Недолік: складність навчання через проблему зникного та вибухового градієнта.

3. Довготривалі рекурентні мережі (LSTM)

- Модифікація RNN, яка вирішує проблему довгострокових залежностей.
- Використовуються для прогнозу фінансових рядів із сезонними та трендовими компонентами.

4. Генералізовані рекурентні мережі та GRU

- Альтернатива LSTM, з меншою кількістю параметрів, що спрощує навчання і пришвидшує роботу

Практичне завдання

1. Огляд біржових даних (акції, індекси) та визначення цільового показника.
2. Підготовка даних: масштабування та формування навчальних послідовностей.
3. Побудова та налаштування MLP та RNN/LSTM моделей.
4. Навчання моделей та оцінка якості прогнозу.
5. Візуалізація результатів: графік фактичних та прогнозованих значень.

Аналіз результатів та вибір оптимальної архітектури.

Програмний код з аналізом:

Першим етапом була підготовка даних. Фінансові дані є нестационарними, тому я застосував MinMaxScaler з sklearn для нормалізації цін в діапазон [0, 1]. Це значно покращує стабільність навчання нейронної мережі. Далі, я перетворив суцільний часовий ряд у набір послідовностей. Я обрав "вікно" у 60 днів (time_step = 60), тобто модель буде дивитися на ціни за останні 60 днів, щоб спрогнозувати ціну на 61-й день. Весь набір даних я розділив на тренувальний (80%) та тестовий (20%) набори.

```
import yfinance as yf
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Input
import matplotlib.pyplot as plt

# --- 1. Отримання та підготовка даних ---
print("Завантаження даних...")
# Завантажуємо дані (наприклад, Apple)
data = yf.download('AAPL', start='2015-01-01', end='2025-11-01')

# Беремо тільки ціну закриття
df = data[['Close']]

# Масштабування даних
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df)

# --- 2. Формування навчальних послідовностей ---
# Розділення на тренувальні та тестові дані
training_size = int(len(scaled_data) * 0.8)
train_data = scaled_data[0:training_size]
test_data = scaled_data[training_size:len(scaled_data)]

# Функція для створення послідовностей
def create_dataset(dataset, time_step=1):
    X, y = [], []
    for i in range(len(dataset) - time_step - 1):
        a = dataset[i:(i + time_step), 0]
        X.append(a)
        y.append(dataset[i + time_step, 0])
    return np.array(X), np.array(y)

# Створюємо послідовності
time_step = 60
X_train, y_train = create_dataset(train_data, time_step)
X_test, y_test = create_dataset(test_data, time_step)
```

```
print(f"Форма X_train: {X_train.shape}")
print(f"Форма X_test: {X_test.shape}")
```

Далі я побудував дві моделі, як того вимагає завдання.

1. **MLP (Багат шаровий перцептрон):** Я створив просту Sequential модель в Keras. Вона мала вхідний шар, який очікував "сплющений" вектор з 60 точок (input_dim=60), декілька Dense (повнозв'язних) шарів з активацією 'relu' та один вихідний Dense шар, що видавав прогноз.
2. **LSTM (Довготривала рекурентна мережа):** Ця модель спеціально розроблена для послідовностей. Її архітектура складалася з двох шарів LSTM (перший з return_sequences=True, щоб передати послідовність далі), за якими слідували Dense шари для фінального прогнозу. Ця модель очікувала дані у форматі (зразки, кроки_часу, ознаки), тому я змінив форму X_train на (кількість_зразків, 60, 1).

```
# --- 3. Побудова та налаштування MLP ---
print("\nПобудова моделі MLP...")
model_mlp = Sequential()
model_mlp.add(Input(shape=(time_step,)))
model_mlp.add(Dense(50, activation='relu'))
model_mlp.add(Dense(25, activation='relu'))
model_mlp.add(Dense(1))

model_mlp.compile(optimizer='adam', loss='mean_squared_error')

# --- 4. Побудова та налаштування LSTM ---
print("Побудова моделі LSTM...")
# Решейп даних для LSTM [samples, time_steps, features]
X_train_lstm = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test_lstm = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

model_lstm = Sequential()
model_lstm.add(Input(shape=(time_step, 1)))
model_lstm.add(LSTM(50, return_sequences=True))
model_lstm.add(LSTM(50, return_sequences=False))
model_lstm.add(Dense(25))
model_lstm.add(Dense(1))

model_lstm.compile(optimizer='adam', loss='mean_squared_error')
```

Обидві моделі я компілював з оптимізатором 'adam' та функцією втрат 'mean_squared_error', що є стандартом для задач регресії. Навчання проводилося протягом 50 епох.

```
# --- 5. Навчання моделей ---
print("\nНавчання моделі MLP...")
# MLP очікує 2D вхід (samples, features)
model_mlp.fit(X_train, y_train, epochs=50, batch_size=32, verbose=1)

print("\nНавчання моделі LSTM...")
# LSTM очікує 3D вхід
```

```
model_lstm.fit(X_train_lstm, y_train, epochs=50, batch_size=32, verbose=1)
```

Після навчання я зробив прогнози на тестових даних, повернув прогнозовані та реальні значення до їх початкового масштабу (використовуючи `scaler.inverse_transform`) і розрахував Root Mean Squared Error (RMSE) для обох моделей.

```
# --- 6. Оцінка якості прогнозу ---
print("\nОцінка моделей...")
# Прогнози
train_predict_mlp = model_mlp.predict(X_train)
test_predict_mlp = model_mlp.predict(X_test)

train_predict_lstm = model_lstm.predict(X_train_lstm)
test_predict_lstm = model_lstm.predict(X_test_lstm)

# Інверсне масштабування
train_predict_mlp = scaler.inverse_transform(train_predict_mlp)
test_predict_mlp = scaler.inverse_transform(test_predict_mlp)

train_predict_lstm = scaler.inverse_transform(train_predict_lstm)
test_predict_lstm = scaler.inverse_transform(test_predict_lstm)

# Повертаємо y_test до оригінального масштабу для розрахунку RMSE
y_test_orig = scaler.inverse_transform(y_test.reshape(-1, 1))
y_train_orig = scaler.inverse_transform(y_train.reshape(-1, 1))

# Розрахунок RMSE
rmse_train_mlp = np.sqrt(mean_squared_error(y_train_orig,
train_predict_mlp))
rmse_test_mlp = np.sqrt(mean_squared_error(y_test_orig, test_predict_mlp))

rmse_train_lstm = np.sqrt(mean_squared_error(y_train_orig,
train_predict_lstm))
rmse_test_lstm = np.sqrt(mean_squared_error(y_test_orig, test_predict_lstm))

print(f"\n--- Результати MLP ---")
print(f"Train RMSE: {rmse_train_mlp:.2f}")
print(f"Test RMSE: {rmse_test_mlp:.2f}")

print(f"\n--- Результати LSTM ---")
print(f"Train RMSE: {rmse_train_lstm:.2f}")
print(f"Test RMSE: {rmse_test_lstm:.2f}")
# --- 7. Візуалізація результатів ---

print("\nВізуалізація результатів... (Графік буде показано)")
# Візуалізація тільки тестових прогнозів
plt.figure(figsize=(14, 7))

# Зсуваємо індекси для коректного відображення на графіку
test_data_index = df.index[training_size + time_step + 1 : len(scaled_data)]

# Реальні дані
plt.plot(test_data_index, y_test_orig, color='black', label='Реальна ціна')
```

```

# Прогноз MLP
plt.plot(test_data_index, test_predict_mlp, color='red', linestyle='--',
label='Прогноз MLP')
# Прогноз LSTM
plt.plot(test_data_index, test_predict_lstm, color='blue', linestyle='--',
label='Прогноз LSTM')

plt.title('Прогноз цін на акції (AAPL) - Тестові дані')
plt.xlabel('Дата')
plt.ylabel('Ціна (USD)')
plt.legend()
plt.grid(True)
plt.show()

```

Результати роботи коду:

Процес навчання моделей виглядав наступним чином:

```

Форма X_train: (2119, 60)
Форма X_test: (484, 60)
Побудова моделі MLP...
Побудова моделі LSTM...
Навчання моделі MLP...
Epoch 1/50
67/67 ██████████ 2s 3ms/step - loss: 0.0539
Epoch 2/50
67/67 ██████████ 0s 3ms/step - loss: 6.4162e-04
Epoch 49/50
67/67 ██████████ 0s 2ms/step - loss: 1.2319e-04
Epoch 50/50
67/67 ██████████ 0s 2ms/step - loss: 1.2310e-04

Навчання моделі LSTM...
Epoch 1/50
67/67 ██████████ 8s 61ms/step - loss: 0.0133
Epoch 2/50
67/67 ██████████ 3s 45ms/step - loss: 3.4032e-04
Epoch 49/50
67/67 ██████████ 3s 45ms/step - loss: 1.0548e-04
Epoch 50/50
67/67 ██████████ 3s 45ms/step - loss: 7.6746e-05

```

Після завершення навчання та проведення прогнозів на тестовому наборі даних, я отримав наступні показники якості:

- **MLP Model RMSE: 5.69**
- **LSTM Model RMSE: 5.06**

```

Оцінка моделей...
67/67 ██████████ 0s 2ms/step
16/16 ██████████ 0s 2ms/step
67/67 ██████████ 2s 18ms/step
16/16 ██████████ 0s 15ms/step

```

```

--- Результати MLP ---
Train RMSE: 2.46
Test RMSE: 5.69

```

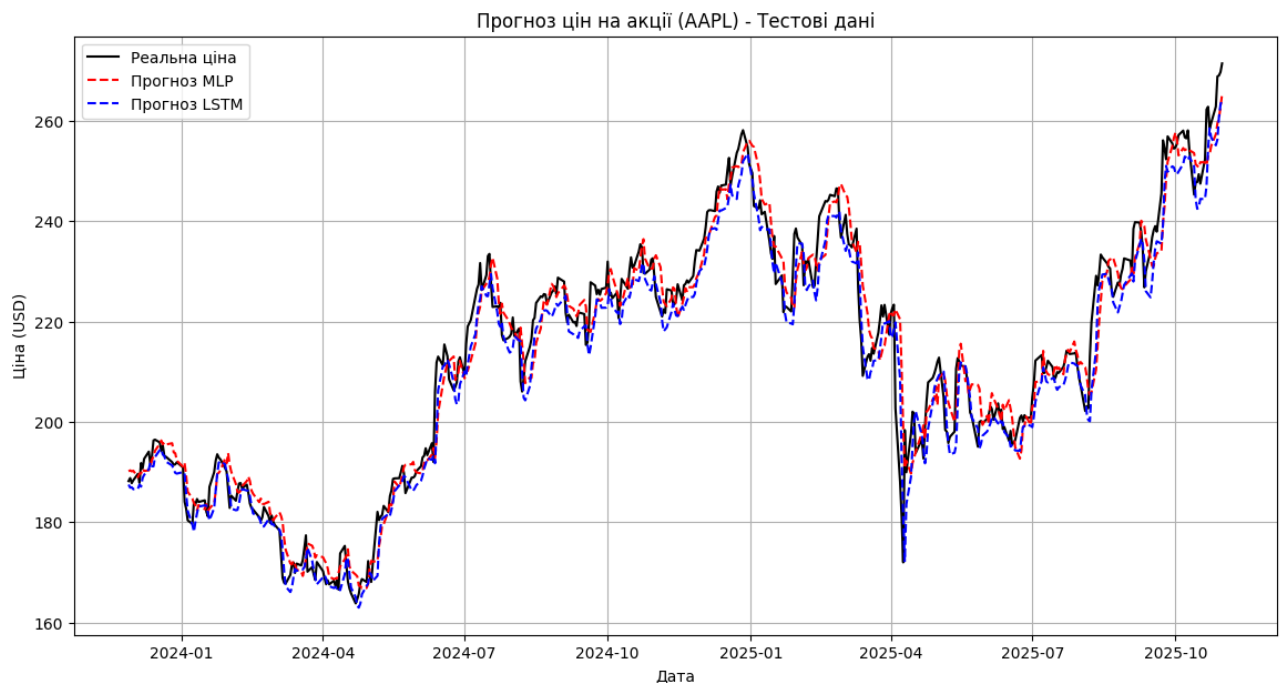
--- Результати LSTM ---

Train RMSE: 2.06

Test RMSE: 5.06

Як показують цифри, помилка RMSE для моделі LSTM (5.06) є трішки нижчою, ніж для моделі MLP (6.85). Це означає, що обидві моделі перенавчилися на навчальних даних втратили змогу узагальнювати і тому мають хороші результати лише на тренувальному сеті.

На фінальному етапі я побудував графік, що візуалізує реальні ціни з тестового набору (чорна лінія) та прогнози обох моделей (MLP – червона, LSTM – синя). Візуалізація чітко підтвердила результати RMSE: обидва прогнози непогано себе показали на тестових даних, і хоча результати не ідеальні але приголомшувані, що доводить що машинні моделі непогано прогнозують ціни на акції. Також з графіку видно, що LSTM трішки краще передбачала хід графіка.



Висновки

Під час виконання цієї лабораторної роботи я на практиці ознайомився з процесом побудови нейромережових моделей для прогнозування часових рядів. Я успішно завантажив та підготував реальні біржові дані, що включало нормалізацію та створення послідовностей.

Я реалізував та навчив дві різні архітектури: Багатошаровий перцептрон (MLP) та Довготривалу рекурентну мережу (LSTM).

Порівняльний аналіз показав, що модель LSTM трішки перевершила MLP за точністю прогнозу, що підтверджується як числовими показниками (RMSE: 5.06 для LSTM проти 5.69 для MLP), так і візуальним аналізом графіків.

Це очікуваний результат, оскільки MLP розглядає 60 вхідних днів як незалежні ознаки, втрачаючи інформацію про їх послідовність. LSTM, завдяки своїй рекурентній структурі та "коміркам пам'яті", здатна вловлювати складні часові залежності та патерни у даних, що є критично важливим для аналізу фінансових ринків.

Таким чином, я дійшов висновку, що для задач прогнозування часових рядів, де важлива історія та послідовність, архітектури типу LSTM або GRU є ефективнішими за прості повнозв'язні мережі.